NTNU | Norwegian University of Science and Technology

Exercise 3

Test planning and execution

TDT4242 Advanced Software Engineering

Group 3
Raffaella Corcione
Ruggero Facchini
Birgitte Louise Vik Thoresen

Spring 2026

Contents

1 Task 3.1. Test Plan **2**
- 1.1 Test Case Table . . . 2
  - 1.1.1 Unit tests . . . 2
  - 1.1.2 Integration tests . . . 7
  - 1.1.3 System tests . . . 7
  - 1.1.4 Non-functional requirement (NFR) tests . . . 8
- 1.2 Coverage Definition . . . 9
- 1.3 Traceability Matrix . . . 9

2 Task 3.2. Unit test execution **13**
- 2.1 Test Implementation . . . 13
- 2.2 Coverage Report . . . 13

3 Task 3.3. system test execution **16**
- 3.1 Requirement list . . . 16
- 3.2 System Test Table . . . 17

4 Task 3.4. Responsible AI Use Reflection **24**
- 4.1 Reflection text . . . 24
  - 4.1.1 Requirements elicitation . . . 24
  - 4.1.2 Web development concepts . . . 24
  - 4.1.3 AI-assisted code generation . . . 24
  - 4.1.4 Revision and correction of AI-generated code . . . 25
  - 4.1.5 Quality assessment of AI-generated code . . . 25
  - 4.1.6 Best practices . . . 25
  - 4.1.7 Professional identity and way of working . . . 26
- 4.2 Attached AI Interaction Log . . . 26
  - 4.2.1 iRE agent . . . 26
  - 4.2.2 GitHub Copilot . . . 33

Chapter 1

Task 3.1. Test Plan

1.1 Test Case Table

1.1.1 Unit tests

Test ID	Test Type	Input / Trigger	Expected Output	Test Method	Pass / Fail Criteria
1	Unit test	Call `validateForm()` with formdata from bulletpoint 1	True	Automated	Function returns boolean `true` with valid fields
2	Unit test	Call `validateForm()` with a very short parameter (bulletpoint 2)	True	Automated	Function returns boolean `true` with valid fields
3	Unit test	Call `validateForm()` with missing parameter (bulletpoint 3)	False	Automated	Function returns boolean `false` and sets error message "Please enter the assignment title"
4	Unit test	Call `validateForm()` with very long parameter (bulletpoint 4)	True	Automated	Function returns boolean `true` with valid fields
5	Unit test	Call `validateForm()` with missing AI tool selection (bulletpoint 5)	False	Automated	Function returns boolean `false` and sets error message "Please select at least one AI tool or enter a custom tool"
6	Unit test	Type = 50 characters in the `cutsomTool` input (bulletpoint 6)	All 50 characters are allowed	Automated	formData.customTool.length <= 50
7	Unit test	Type > 50 characters in the `cutsomTool` input (bulletpoint 7)	Only the first 50 characters are allowed	Automated	formData.customTool.length > 50
8	Unit test	Call `validateForm()` with `cutsomTool` with a special character (bulletpoint 8)	False	Automated	Function returns boolean `false` and sets error message "Custom tool name contains invalid characters"
9	Unit test	Call `handleFileChange()` with a non-image file (bulletpoint 9)	False	Automated	Function sets error message "Please upload an image file"
10	Unit test	Call `handleFileChange()` with image file > $5MB$ (bulletpoint 10)	False	Automated	Function sets error message "File size must be less than 5MB"

Table 1.1: Client Unit Test Cases for AIGuidebook System

1. Normal functioning of the declaration form:
    - Name: Mario

- Assignment title: Assignment 1
- AI tools: ChatGPT
- Other tool: none
- Purpose: summary of lecture contents
- Content: this is an example content.
- Screenshot: none

2. Declaration form boundary value test, very short parameter:

   - Name: M
   - Assignment title: Assignment 1
   - AI tools: ChatGPT
   - Other tool: none
   - Purpose: summary of lecture contents
   - Content: this is an example content.
   - Screenshot: none

3. Declaration form boundary value test, missing parameter:

   - Name: Mario
   - Assignment title: none
   - AI tools: ChatGPT
   - Other tool: none
   - Purpose: summary of lecture contents
   - Content: this is an example content.
   - Screenshot: none

4. Declaration form boundary value test, very long parameter:

   - Name: Mario
   - Assignment title: Assignment 1
   - AI tools: ChatGPT
   - Other tool: none
   - Purpose: AI Guidebook for Students is a student-centred web application developed to support everyday academic work in an era where generative AI tools such as ChatGPT, GitHub Copilot, and Claude are increasingly embedded in study practices. As students adopt AI tools rapidly, there is a growing need for clear guidance that helps them use AI responsibly, transparently, and in alignment with institutional academic integrity policies. The AI Guidebook provides a structured environment where students can reflect on, document, and manage their AI use while remaining compliant with university-level ethical and academic standards. Functionally, the system shall offer AI

usage logging, assignment-level AI declarations, ethical guidelines, and automated compliance checks against institutional rules (e.g., NTNU), combined with a personal dashboard that visualizes usage patterns over time. From a non-functional perspective, the application emphasizes transparency, explainability, privacy-aware data handling, and usability, ensuring that AI support strengthens learning rather than replacing student responsibility.

- Content: this is an example content.
- Screenshot: none

5. Declaration form boundary value test, missing AI tool selection:
   - Name: Mario
   - Assignment title: Assignment 1
   - AI tools: none
   - Other tool: none
   - Purpose: this is an example purpose.
   - Content: this is an example content.
   - Screenshot: none
6. Optional AI tool text input test, boundary value test, 50 character string:
   - Name: Mario
   - Assignment title: Assignment 1
   - AI tools: ChatGPT
   - Other tool: 12345678901234567890123456789012345678901234567890
   - Purpose: This is an example purpose.
   - Content: This is an example content.
   - Screenshot: none
7. Optional AI tool text input test, boundary value test, more than 50 character string:
   - Name: Mario
   - Assignment title: Assignment 1
   - AI tools: ChatGPT
   - Other tool: 123456789012345678901234567890123456789012345678901234567890
   - Purpose: This is an example purpose.
   - Content: This is an example content.
   - Screenshot: none
8. Optional AI tool text input test, boundary value test, string with special characters:
   - Name: Mario

- Assignment title: Assignment 1
- AI tools: ChatGPT
- Other tool: π
- Purpose: This is an example purpose.
- Content: This is an example content.
- Screenshot: none

9. File input, boundary value test, not an image file:
   - Name: Mario
   - Assignment title: Assignment 1
   - AI tools: ChatGPT
   - Other tool: none
   - Purpose: This is an example purpose.
   - Content: This is an example content.
   - Screenshot: IEEE-SRS-830-1998.pdf
10. File input, boundary value test, image file larger than 5MB:
    - Name: Mario
    - Assignment title: Assignment 1
    - AI tools: ChatGPT
    - Other tool: none
    - Purpose: This is an example purpose.
    - Content: This is an example content.
    - Screenshot: danny-greenberg-HAP8ECXx5CA-unsplash.jpg

Test ID	**Test Type**	**Input / Trigger**	**Expected Output**	**Test Method**	**Pass / Fail Criteria**
11	Unit test	POST /api/declarations with userName missing (bullet-point 1)	400 validation error	Automated	Returns error: "All fields except screenshot are required"
12	Unit test	POST /api/declarations with tool names > 50 characters (bulletpoint 2)	400 validation error	Automated	Returns error: "AI tools must be a non-empty array with each tool name not exceeding 50 character"
13	Unit test	POST /api/declarations with aiTools = [] (bulletpoint 3)	400 validation error	Automated	Returns error: "AI tools must be a non-empty array with each tool name not exceeding 50 character"
14	Unit test	POST /api/declarations with a file that is not an image (bulletpoint 4)	400 validation error	Automated	Returns error: "Only image files are allowed (jpeg, jpg, png, gif, webp)"
15	Unit test	POST /api/declarations with a file $> 5MB$	400 validation error	Automated	multer triggers a limit filesize error.

Table 1.2: Server Unit Test Cases for AIGuidebook System

Server components: server-side input validation

11. Form input validation, boundary value test, missing parameters:
    - Name: none
    - Assignment title: Assignment 1
    - AI tools: ChatGPT
    - Other tool: none
    - Purpose: This is an example purpose.
    - Content: This is an example content.
    - Screenshot: none
12. Form input validation, boundary value test, string with special characters in custom AI tool text field:
    - Name: Mario
    - Assignment title: Assignment 1
    - AI tools: ChatGPT
    - Other tool: tool π 314
    - Purpose: This is an example purpose.
    - Content: This is an example content.
    - Screenshot: none
13. Form input validation, missing AI tool selection:
    - Name: Mario
    - Assignment title: Assignment 1
    - AI tools: none
    - Other tool: none
    - Purpose: this is an example purpose.
    - Content: this is an example content.
    - Screenshot: none
14. Screenshot upload validation, not an image file:
    - Name: Mario
    - Assignment title: Assignment 1
    - AI tools: ChatGPT
    - Other tool: none
    - Purpose: This is an example purpose.
    - Content: This is an example content.
    - Screenshot: IEEE-SRS-830-1998.pdf

15. Screenshot upload validation, image larger than 5MB:

    - Name: Mario
    - Assignment title: Assignment 1
    - AI tools: ChatGPT
    - Other tool: none
    - Purpose: This is an example purpose.
    - Content: This is an example content.
    - Screenshot: danny-greenberg-HAP8ECXx5CA-unsplash.jpg

1.1.2 Integration tests

Test ID	Test Type	Input / Trigger	Expected Output	Test Method	Pass / Fail Criteria
16	Integration test	Submission to the declaration form with all valid fields	Correct storing in database	Automated	Db contains the new submission
17	Integration test	Submission to the declaration form of a screenshot	Storage of the submission with the screenshot file	Automated	Screenshot file is correctly stored
18	Integration test	Submission to the declaration form with some incorrect parameters	db state is unchanged	Automated	Same records as before in db
19	Integration test	Client startup when server not running	Display of error message	Automated	Correct error handling

Table 1.3: Integration Test Cases for AIGuidebook System

16. Typical workflow test: normal form submission with correct parameters. The test will assert that the submission is correctly stored in the database. The test will ensure correct functioning of frontend-backend-database communication.

17. Screenshot upload test: normal form submission including the screenshot. The test will assert that the screenshot is correctly stored.

18. Incorrect form submission: form submission with incorrect parameters. The test will assert that the server state is unchanged. Note: under normal functioning of the application, a wrong submission will never reach the server, so this test is equivalent to the unit tests performed on client-side validation.

19. Error handling: start up of the client application with the server not running. The test will assert the correct display of the error message on the client interface.

1.1.3 System tests

20. Normal user workflow: fill in a declaration form with all valid parameters and click "submit declaration". After the submission, the user is redirected to the home page that correctly displays the new declaration.

Test ID	Test Type	Input / Trigger	Expected Output	Test Method	Pass / Fail Criteria
20	System test	Submit declaration form with valid fields (name, assignment title, AI tool selection, purpose, content)	Submission is accepted	Manual	Redirects to homepage, and submission is stored in the db
21	System test	Submit declaration form with specified tools used from a list	Selected tools appear in the declaration summary	Manual	Selected tools is stored successfully and loads correctly on the homepage
22	System test	Submit declaration form with a valid custom tool	Custom tool appear in the declaration summary	Manual	Custom tool is stored successfully and loads correctly on the homepage
23	System test	Submit declaration form with AI-generated content specified	AI-generated content appear in the declaration summary	Manual	AI-generated content is stored successfully and loads correctly on the homepage
24	System test	Submit declaration form with a valid screenshot	Uploaded screenshot is displayed as an image in the declaration summary	Manual	Uploaded screenshot is stored successfully and loads correctly on the homepage

Table 1.4: System Test Cases for AIGuidebook System

21. Selection of AI tools from the list: the user fills in a declaration selecting one or more tools from the list and submits it. The records are stored correctly in the database and shown to the user as a single declaration.

22. Submission of a custom AI tool: the user submits a declaration that contains a custom tool entered through the dedicated text input field. The submission is stored correctly and displayed to the user in the home page.

23. Submission of a screenshot: the user submits a declaration that contains an image for upload. After form submission, the image is correctly displayed along with the new declaration in the home page.

1.1.4 Non-functional requirement (NFR) tests

The following tests ensure proper functionality of the database architecture according to our non-functional requirements.

Test ID	Test Type	Input / Trigger	Expected Output	Test Method	Pass / Fail Criteria
25	NFR test	Populate db with 10,000 logged AI usage records and interact with the system as normal	System works fully within acceptable performance and smooth UI responses	Manual / Automated	System renders without crashing or delays
26	NFR test	Retrieve, update and delete declarations saved in the db	Data is stored reliably, updates is correctly changed and deleted records cannot be retrieved	Manual	Retrieve the correct stored data, update correctly changed fields and delete removes the record permanently

Table 1.5: System Test Cases for AIGuidebook System

1.2 Coverage Definition

All the functional requirements are covered by at least one test (see traceability matrix). The main code files involved in the tests are the two frontend pages (HomePage.tsx, DeclarationFormPage.tsx) and the backend page (declarations.js). For this code, we aim to achieve a line coverage of 60% and a branch coverage of 60%.

1.3 Traceability Matrix

We provide the traceability matrix for our unit and system tests. As the tables show, our test plan provides 100% requirements coverage.

Test n.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
FR3	x	x	x	x	x	x	x	x			x	x	x		
FR4	x				x	x	x	x				x	x		
FR5	x				x								x		
FR6						x	x	x				x			
FR7	x														
FR8									x	x				x	x

Table 1.6: Unit tests requirement coverage

1. Test 1 covers the method `validateForm()` (lines 117-150) in `files/client/src/pages/DeclarationFormPage.tsx`.
2. Test 2 covers the method `validateForm()` (lines 117-150) in `files/client/src/pages/DeclarationFormPage.tsx`. For example, a test with `formData.userName` submitted as a very short parameter will be covered in lines 118-121.
3. Test 3 covers the method `validateForm()` (lines 117-150) in `files/client/src/pages/DeclarationFormPage.tsx`. For example, a test with `formData.assignmentTitle` submitted with a missing parameter will be covered in lines 122-125.
4. Test 4 covers the method `validateForm()` (lines 117-150) in `files/client/src/pages/DeclarationFormPage.tsx`. For example, a test with `formData.usagePurpose` submitted as a very long parameter will be covered in lines 141-144.
5. Test 5 covers the section for `formData.aiTools` in the method `validateForm()` (lines 126-129) in `files/client/src/pages/DeclarationFormPage.tsx`.
6. Test 6 covers the element `formData.customTool` (line 308) in `files/client/src/pages/DeclarationFormPage.tsx`.
7. Test 7 covers the element `formData.customTool` (line 308) in `files/client/src/pages/DeclarationFormPage.tsx`.
8. Test 8 covers the section for `formData.customTool` in the method `validateForm()` (lines 134-138) in `files/client/src/pages/DeclarationFormPage.tsx`.

9. Test 9 covers the section for `file.type` in the method `handleFileChange()` (lines 77-80) in `files/client/src/pages/DeclarationFormPage.tsx`.

10. Test 10 covers the section for `file.size` in the method `handleFileChange()` (lines 83-86) in `files/client/src/pages/DeclarationFormPage.tsx`.

11. Test 11 covers the method `router.post()` (lines 87-101) in `files/server/router/declarations.js`.

12. Test 12 covers the method `router.post()` (lines 107-109) in `files/server/router/declarations.js`.

13. Test 13 covers the method `router.post()` (lines 107-109) in `files/server/router/declarations.js`.

14. Test 14 covers the method `fileFilter()` (lines 31-41) in `files/server/router/declarations.js`.

15. Test 15 covers the method `upload()` (line 46) in `files/server/router/declarations.js`.

Test n.	16	17	18	19
FR3	x		x	x
FR4	x			
FR5				
FR6				
FR7				
FR8		x		

Table 1.7: Integration tests requirement coverage

16. Test 16 covers the workflow of form submission from `files/client/src/pages/DeclarationFormPage.tsx` (method `handleSubmit()`, lines 155-208) to `files/server/router/declarations.js` (method `router.post()`, lines 86-177).

17. Test 17 covers workflow of the form submission including a screenshot upload from `files/client/src/pages/DeclarationFormPage.tsx` (method `handleFileChange()`, lines 73-101) to `files/server/router/declarations.js` (method `upload()`, lines 43-47).

18. Test 18 covers the workflow of incorrect form submission scenarios, from `files/client/src/pages/DeclarationFormPage.tsx` (method `validateForm()`, lines 117-150) to `files/server/router/declarations.js` (method `router.post()`, lines 86-177).

19. Test 19 covers the workflow of client startup when server not running, in `files/client/src/pages/HomePage.tsx` (method `fetchDeclarations()`, lines 27-44).

Test n.	20	21	22	23	24
FR3	x	x	x	x	x
FR4		x	x		
FR5		x			
FR6			x		
FR7				x	
FR8					x

Table 1.8: System tests requirement coverage

20. Test 20 covers the normal user workflow of submitting a declaration form with all valid fields in `files/client/src/pages/DeclarationFormPage.tsx` (method `handleSubmit()` lines 155-208) and verifying that the declarations is displayed on the homepage, `files/client/src/pages/HomePage.tsx`.

21. Test 21 covers submitting a declaration form with specified tools selected from the provided list in `files/client/src/pages/DeclarationFormPage.tsx` (method `handleSubmit()` lines 155-208) and verifying that the selected tools appear correctly in the summary on the homepage, `files/client/src/pages/HomePage.tsx`.

22. Test 22 covers submitting a declaration form with a custom tool in `files/client/src/pages/DeclarationFormPage.tsx` (method `handleSubmit()` lines 155-208) and verifying that the custom tool appear correctly in the summary on the homepage, `files/client/src/pages/HomePage.tsx`.

23. Test 23 covers submitting a declaration form with AI-generated content specified in `files/client/src/pages/DeclarationFormPage.tsx` (method `handleSubmit()` lines 155-208) and verifying that the specified AI-generated content appear correctly in the summary on the homepage, `files/client/src/pages/HomePage.tsx`.

24. Test 24 covers submitting a declaration form with a valid screenshot upload in `files/client/src/pages/DeclarationFormPage.tsx` (method `handleSubmit()` lines 155-208) and verifying that the uploaded image is displayed correctly in the summary on the homepage, `files/client/src/pages/HomePage.tsx`.

Test n.	25	26
NFR3	x	x
NFR5	x	x

Table 1.9: Non-functional requirement coverage

25. Test 25 covers system performance when interacting with a large dataset. The test populates the database with 10,000 AI usage declarations and exercises the normal workflows (submission in `files/client/src/pages/DeclarationFormPage.tsx` via `handleSubmit()` and display on `files/client/src/pages/HomePage.tsx`).

26. Test 26 covers retrieval, updating, and deletion of declarations of the backend API for administrative operations. It tests the `POST /api/declarations` and `GET /api/declarations` endpoints implemented in `files/server/routes/declarations.js`, as well as deleting declarations in the database.

Chapter 2

Task 3.2. Unit test execution

2.1 Test Implementation

All the unit tests are available at `https://github.com/Raffaella-Corcione/TDT4242_group03` under `files/tests`. The tests reflect the test plan defined in task 3.1 and they were generated with the help of GitHub Copilot.

To run the tests locally locate to the `files` folder, install dev dependencies as described in the `README.md` file of the repository. Also in the `files` folder, use `npm run test` to run the tests. To run with coverage, run `npm run coverage`.

2.2 Coverage Report

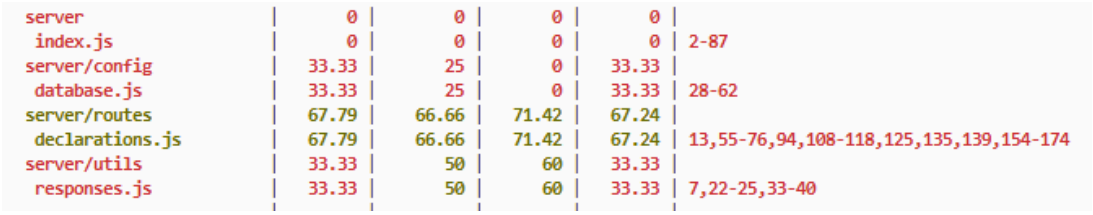
After generating the tests as described in Task 3.1, the overall client- and server-side coverage did not meet the coverage report requirements. In the task description the objective was to, "achieve high structural test coverage, specifically: Line coverage: $\geq 60\%$ Branch coverage: $\geq 70\%$ Function/method coverage: $\geq 80\%$".

On the client side there were no tests for `HomePage.tsx` and `App.tsx`, and `DeclarationFormPage.tsx` showed insufficient function coverage (as shown in Figure 2.1).

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	55.73	51.66	44	55.55	
src	0	100	0	0	
App.tsx	0	100	0	0	14-15
src/pages	56.3	51.66	45.83	56.14	
DeclarationFormPage.tsx	75.28	73.8	64.7	76.19	63,86-97,105-109,117-118,140-141,144-145,181,195-199,212
HomePage.tsx	0	0	0	0	15-146
src/types	100	100	100	100	
index.ts	100	100	100	100	

Figure 2.1: Initial client-side coverage

On the server side, there were initially no unit tests for `index.js`, and only limited coverage of `database.js` and `responses.js`, as well as insufficient function and lines coverage in `declarations.js` as shown in Figure 2.2.



server	0	0	0	0	
index.js	0	0	0	0	2-87
server/config	33.33	25	0	33.33	
database.js	33.33	25	0	33.33	28-62
server/routes	67.79	66.66	71.42	67.24	
declarations.js	67.79	66.66	71.42	67.24	13,55-76,94,108-118,125,135,139,154-174
server/utils	33.33	50	60	33.33	
responses.js	33.33	50	60	33.33	7,22-25,33-40

Figure 2.2: Initial server-side coverage

Client-side

On the client-side there was added some unit tests for `DeclarationFormPage.tsx`, `HomePage.tsx` and `App.tsx`. These test were generated with the help of GitHub Copilot as well.

1. **DeclarationFormPage.cancel-navigate-home.test.tsx** verifies that clicking *Cancel* returns the user to the home page.
2. **DeclarationFormPage.file-preview-and-remove.test.tsx** verifies that a valid image shows a preview and that the preview disappears after *Remove*
3. **DeclarationFormPage.submit-failure.test.tsx** verifies that a failed submission displays the server's error message to the user.
4. **DeclarationFormPage.submit-success.test.tsx** verifies that a successful submission redirects the user to the home page.

1. **HomePage.empty-state.test.tsx** verifies that an empty result set shows the *No declarations yet* message and a count of zero.
2. **HomePage.error-state.test.tsx** verifies that a failed fetch shows an error banner and hides the loading indicator.
3. **HomePage.loading-state.test.tsx** verifies that the loading message appears while declarations are being fetched.
4. **HomePage.retry-refetch.test.tsx** verifies that pressing *Retry* after an error triggers a new fetch and then displays the loaded data.
5. **HomePage.success-grouping.test.tsx** verifies that multiple entries for the same assignment are grouped together and that key details (tools, user, date, purpose, content) are shown.

1. **App.link-navigation.test.tsx** verifies that clicking *Create New Declaration* navigates to the declaration form page.
2. **App.new-declaration-route.test.tsx** verifies that the declaration form page renders when the route is `/new-declaration`.
3. **App.root-route.test.tsx** verifies that the home page renders when the route is `/`.

Server-side

On the server side, some tests were added to test the failure handling more exhaustively. Special attention was paid to the database failures:

1. **declarations.dbfailure.test.ts** tests the two instances in which a serverError could be called: a GET request failure in fetching the declarations and a POST request failure in submitting a new declaration.

The server tests use Vite to mock the original express, multer and database modules. This approach allowed us to make focused tests, but it was also very difficult to implement. For this reason, we do not have as many tests for the server as we have for the client components.

Resulting Coverage Report

When running `npm run coverage` after the new unit test was added, this resulted in the test coverage as shown in the Figure 2.3.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	72.57	73.38	78.57	71.86	
client/src	100	100	100	100	
App.tsx	100	100	100	100	
client/src/pages	90.75	83.33	91.66	90.35	
DeclarationFormPage.tsx	88.76	80.95	88.23	88.09	63,117-118,140-141,144-145,181,195,212
HomePage.tsx	96.66	88.88	100	96.66	36
client/src/types	100	100	100	100	
index.ts	100	100	100	100	
server	0	0	0	0	
index.js	0	0	0	0	2-87
server/config	33.33	50	0	33.33	
database.js	33.33	50	0	33.33	28-62
server/routes	83.05	73.33	100	82.75	
declarations.js	83.05	73.33	100	82.75	13,65,94,108-118,125,154,166
server/utils	77.77	78.57	60	77.77	
responses.js	77.77	78.57	60	77.77	7,33

Figure 2.3: Final coverage report

Chapter 3

Task 3.3. system test execution

This task includes system test execution for codebase developed by Group 02. The working codebase is found at `https://github.com/Evangaard/TDT4242-ai-guidebook`.

3.1 Requirement list

- FR-1.1 Manual usage entry: As a student, I want to manually log my AI tool usage by entering the tool name, assignment ID, usage duration, and context so that I can maintain an accurate record of my AI assistance for each assignment.
- FR-1.3 Usage categorization: As a student, I want to categorize my AI usage by task type (brainstorming, drafting, summarizing, code assistance, editing) so that I can accurately represent how I used AI tools in my academic work.
- FR-2.1 Assignment-level declaration: As a student, I want to submit an AI usage declaration that includes assignment ID, AI tools used, usage context, duration, and compliance confirmation so that I can formally document my AI assistance in accordance with institutional policies.
- FR-2.2 Declaration amendment: As a student, I want to edit my submitted declarations with a mandatory reason field and maintain version history so that I can correct errors or add forgotten information while maintaining transparency.
- FR-3.1 Usage overview dashboard: As a student, I want to view my total AI usage time, frequency by tool, and task breakdown on a personal dashboard so that I can understand my AI usage patterns over time.
- FR-3.2 Compliance status display: As a student, I want to see whether my AI usage aligns with NTNU institutional guidelines for each assignment so that I can verify I am following academic integrity policies.
- FR-3.3 Comparative analytics: As a student, I want to compare my usage time, frequency, and tool selection against anonymized peer averages so that I can contextualize my AI usage relative to other students.

- NFR-1 Learnability: As a student, I want to successfully complete my first AI usage log entry within 7 minutes of first system access without external help so that the system does not require extensive training to use effectively.
- NFR-2 Task efficiency: As a student, I want to complete routine AI usage logging in under 5 minutes per session using pre-filled templates so that documentation does not become a time burden during my study workflow.

3.2 System Test Table

Test ID	Requirement ID	Test Scenario	Pass/Fail
1	FR-1.1	Assignment declaration	Pass
2	FR-1.3	AI usage categorization	Pass
3	FR-2.1	Compliance confirmation check during declaration	Pass
4	FR-2.2	Declaration editing	Pass
5	FR-3.1	Usage overview dashboard data breakdown	Pass
6	FR-3.2	Compliance status display	Pass
7	FR-3.3	Comparing with anonymized peer student data	Fail
8	NFR-1	Time to complete first log entry	Pass
9	NFR-2	Routine usage logging with pre-filled templates	Fail

Table 3.1: System Test Cases for AIGuidebook System

1. Test 1
    - Input/Steps: created a new assignment declaration though the button "Log New AI Usage" with the following parameters:
        - AI tool: Claude
        - Assignment ID: assignment 4
        - Duration: 30
        - Usage Context: making mind maps of geography homework
        - Task Type: summarizing

      the press the "Log usage" at the end of the form.
    - Expected Result: the declaration is successfully created and saved in the database.
    - Actual Result: the created declaration is present on homepage under " usage history".
    - Evidence: see the screenshots below.
2. Test 2
    - Input/Steps: created a new assignment declaration though the button "Log New AI Usage" with the following parameters:
        - AI tool: GitHub Copilot

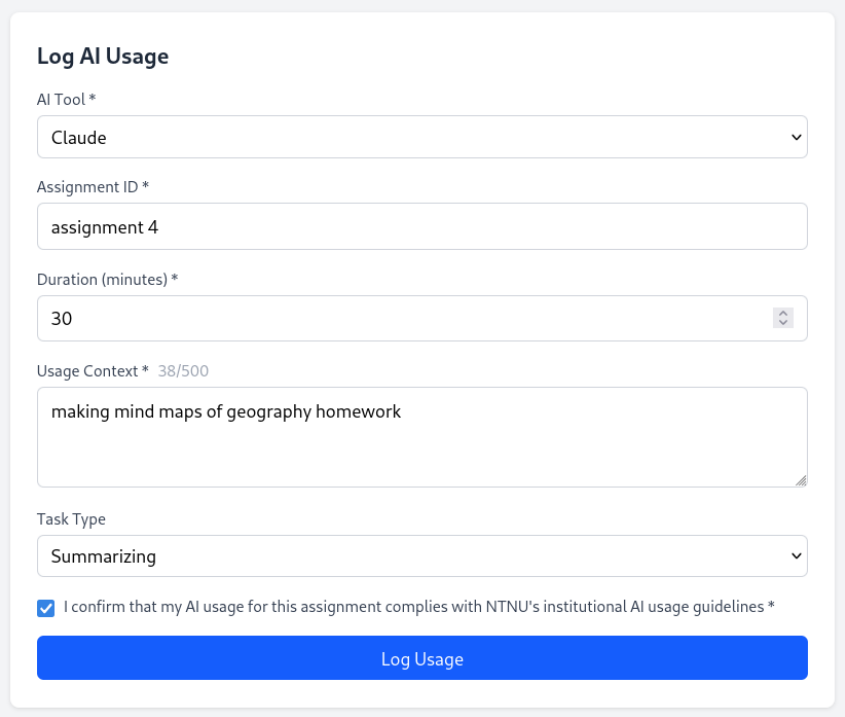


Figure 3.1: Test 1: declaration form

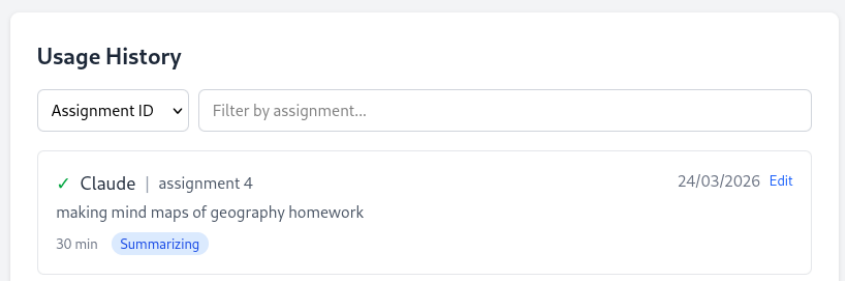


Figure 3.2: Test 1: record of newly created assignment declaration

  - Assignment ID: assignment 5
  - Duration: 90
  - Usage Context: making mind maps of geography homework
  - Task Type: Code Assistance

  the press the "Log usage" at the end of the form.
- Expected Result: the declaration is created with a task type from a predefined list to categorize AI usage type.
- Actual Result: the submitted declarations have their task type visible on the homepage.
- Evidence: see the screenshots below.

3. Test 3

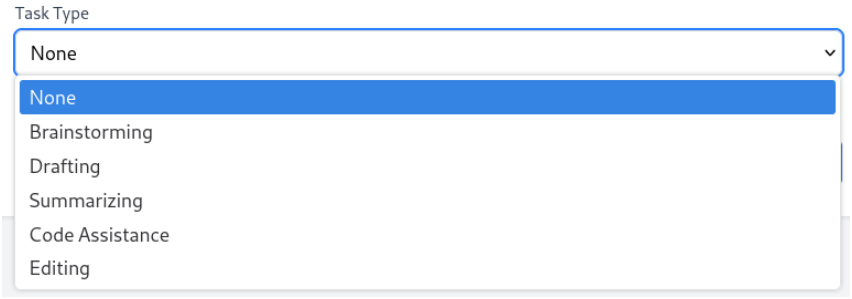


Figure 3.3: Test 2: task type categorization

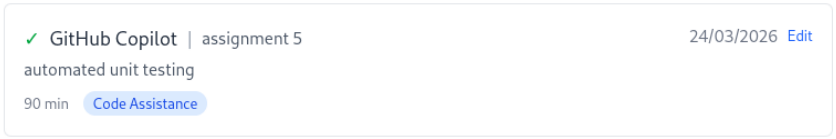


Figure 3.4: Test 2: a declaration containing its task type

- Input/Steps: created a new assignment declaration though the button "Log New AI Usage" with the following parameters:
  - AI tool: GitHub Copilot
  - Assignment ID: assignment 5
  - Duration: 90
  - Usage Context: making mind maps of geography homework
  - Task Type: Code Assistance

  then check of the mandatory checkbox for compliance. Then pressed the "Log usage" at the end of the form to submit.
- Expected Result: The declaration is created with compliance confirmation.
- Actual Result: The declaration is visible in the homepage with a green check mark when the compliance is confirmed.
- Evidence: see the screenshots below.

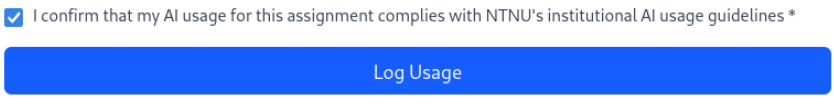


Figure 3.5: Test 3: compliance confirmation check

4. Test 4

- Input/Steps: created a new assignment declaration though the button "Log New AI Usage" with the following parameters:
  - AI tool: Gemini
  - Assignment ID: assignment 3

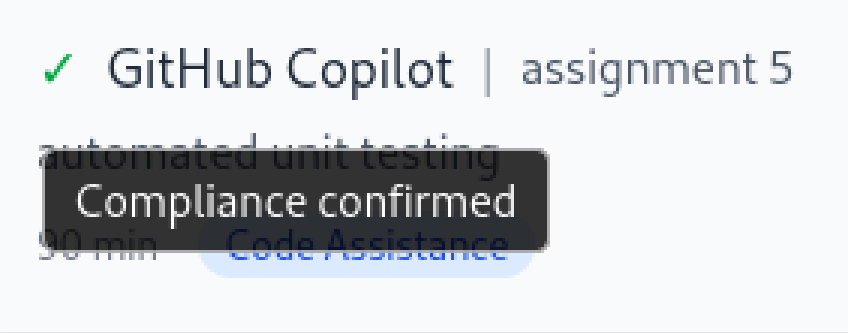


Figure 3.6: Test 3: compliance check mark for submitted declaration

  - Duration: 30
  - Usage Context: making mind maps of geography homework
  - Task Type: summarizing

  after that, clicked on the "Edit" button on the newly created assignment. Modified the form values by writing in Usage Context "making mind maps of geography homework and generating images." and "forgot to include one usage context." as Reason for Amendment. Clicked on "Update Entry".
- Expected Result: editing my submitted declaration with a mandatory reason field and maintaining version history.
- Actual Result: the modified declaration appears in the "Usage History" page with the viewable Amendment History.
- Evidence: see screenshots below.

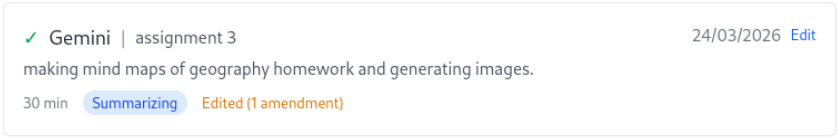


Figure 3.7: Test 4: modified declaration

5. Test 5

- Input/Steps: accessed the dashboard with some existing declarations present and displayed. Assessed the presence of the following items:
  - total AI usage time
  - AI usage frequency by tool
  - AI usage breakdown by task
- Expected Result: I want to view my total AI usage time, frequency by tool, and task breakdown on a personal dashboard so that I can understand my AI usage patterns over time.
- Actual Result: all the required information is present and clearly displayed on the dashboard page, which is also the page that loads at the start of the application.

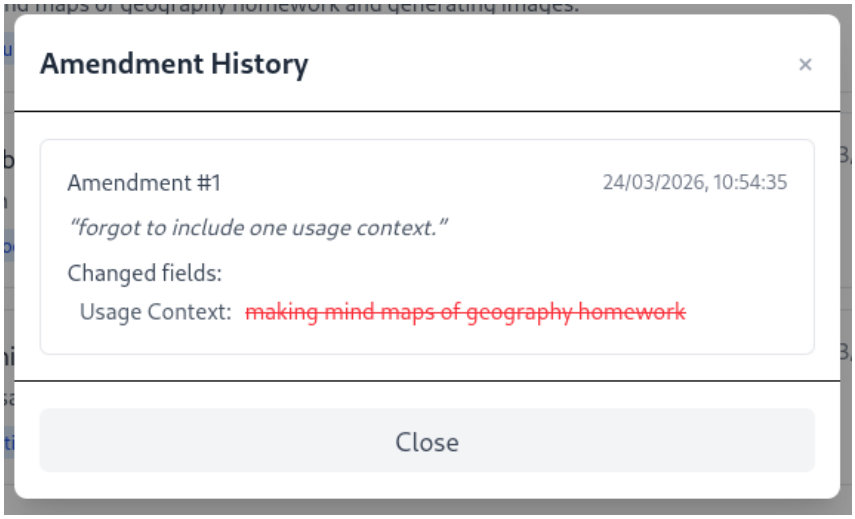


Figure 3.8: Test 4: amendment history

- Evidence: see screenshots below.

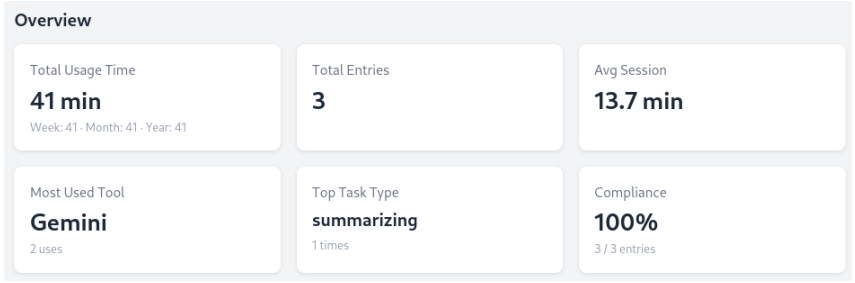


Figure 3.9: Test 5: dashboard overview

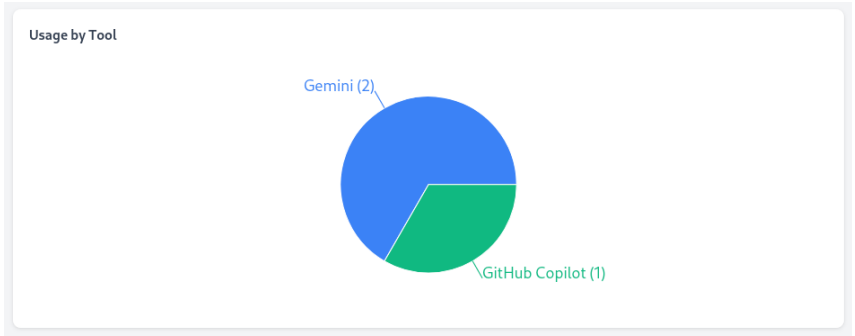


Figure 3.10: Test 5: AI usage tool breakdown chart

6. Test 6

- Input/Steps: accessed the dashboard with some existing declarations present and displayed. Assessed the presence of the following items:
  - Compliance status information

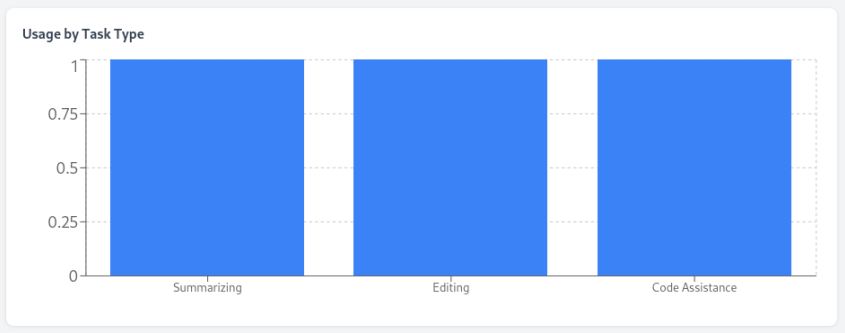


Figure 3.11: Test 5: AI task type breakdown chart

- Expected Result: want to see whether my AI usage aligns with NTNU institutional guidelines for each assignment so that I can verify I am following academic integrity policies.
- Actual Result: all the required information is present and clearly displayed on the dashboard page. Note: the assignment declaration process requires checking a field "I confirm that my AI usage for this assignment complies with NTNU's institutional AI usage guidelines" in order to complete the declaration. As a consequence of this, it seems that compliance of 100% on all declarations is the only possible status.
- Evidence: see screenshots below.

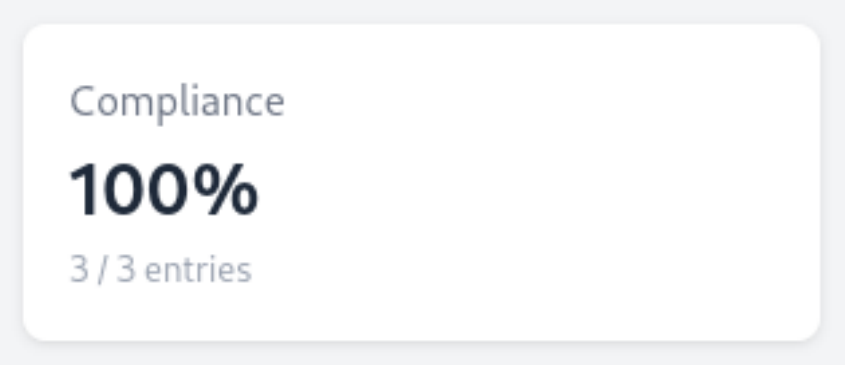


Figure 3.12: Test 6: compliance information on declared assignments

7. Test 7

- Input/Steps: accessed the dashboard with some existing declarations present and displayed. Assessed the presence of the following items:
  - peer comparison tab with data from other students about usage time, frequency and tools selection.
- Expected Result: want to compare my usage time, frequency, and tool selection against anonymized peer averages so that I can contextualize my AI usage relative to other students.

- Actual Result: The dashboard page contains an element titled "Peer Comparison". Since the application does not require login of the users or other identification of the students submitting different declarations (at least in a way that was evident to a user and testable in a local environment), it was not possible to achieve a multiple user scenario.
- Evidence: see screenshots below.

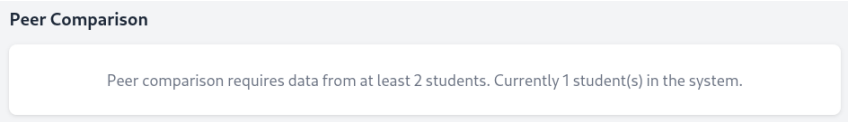


Figure 3.13: Test 7: peer comparison tab

8. Test 8
    - Input/Steps: same as test 1. Additionally, measure the time used to complete the task.
    - Expected Result: successfully complete the first AI usage log entry within 7 minutes.
    - Actual Result: while executing test 1, the time to complete the log entry was approximately 1 minute.
    - Evidence: there is no observable evidence for this test.
9. Test 9
    - Input/Steps: same as test 8.
    - Expected Result: successfully complete the AI usage log entry within 5 minutes using pre-filled templates.
    - Actual Result: there were no pre-filled templates visible on the application interface. However, the time to fill in the declaration was still under 5 minutes.
    - Evidence: there is no observable evidence for this test.

Chapter 4

Task 3.4. Responsible AI Use Reflection

4.1 Reflection text

4.1.1 Requirements elicitation

The iRE Agent provided answers that were clear, organized, and complete, making it easy to translate them into requirements. However, the agent often gave pre-prepared information, so more specific questioning was needed to extract the right details. Human stakeholders, in contrast, gave answers that were sometimes unclear or not directly usable, but they included more context, ideas, and details that the AI could not provide. Interviews with humans required more follow-up questions, yet they revealed real needs and perspectives that added depth to the requirements. Overall, the AI was faster and more precise, while humans offered richer insight and a more natural understanding of the problem.

4.1.2 Web development concepts

AI tools were helpful in understanding web development concepts, architectures, and framework. They provided clear explanations of code structure and suggested good architecture choices for the project, making it easier to see how different parts of the system fit together. Even without prior knowledge of a specific framework, the AI could give a solid understanding of how it works and how to use it effectively. However, some explanations still required manual validation, to make sure they were correct and could be used in the project.

4.1.3 AI-assisted code generation

We produced the prompt without AI assistance, following the practices from the lectures and documented in Task 2.1. The strategy combined role-based prompting with a chain-of-thought approach, requiring the model to reason step by step and produce one file at a time. All functional and non-functional requirements were included, together with the exact database schema and technical constraints on file storage and input validation. This

level of detail proved effective, where the model generated a working full-stack application in a single session.

4.1.4 Revision and correction of AI-generated code

Code review was carried out manually by the team, with AI tools used occasionally to verify specific details. The application worked as expected on first execution, so the review focused on correctness and consistency rather than debugging. The main issue found was a logical defect in the declaration counter on the homepage. The generated code used the raw database row count instead of the number of distinct submission groups, fixed by replacing it with the key count of the already-computed groupedDeclarations object. The review also led to extracting inline response construction into a centralised utility module.

We also employed the model for automated test case generation. This activity, though still done with careful prompting, proved to be less efficient than the code generation. Generating tests that worked well took many tries and sometimes we had to explicitly tell the model which mistakes to avoid and what to pay special attention to. This process considerably increased the amount of time needed to complete the task.

4.1.5 Quality assessment of AI-generated code

The code produced by GitHub Copilot was of generally good quality. Correctness was strong and all required features worked on first run, with only one logical defect found. Readability was a clear strength, with JSDoc comments on every function, consistent naming, and systematic use of TypeScript types. Structurally, the codebase followed a clean three-tier separation, further improved by the response utility module introduced during review. Maintainability was adequate, though generated files were on the longer side and the absence of linting would need to be addressed before further development. Alignment with requirements was complete across all functional and non-functional requirements. Overall, the code needed only minor adjustments and served as a solid foundation.

4.1.6 Best practices

The prevalent practices we adopted in our interactions with AI were role-based prompting and guided reasoning. These practices helped us obtain professional quality code, as well as clear and properly documented explanations. The responses we obtained were focused and competent, which indicates that we were able to effectively guide the model towards the goal. As far as role-based prompting is concerned, we instructed Copilot to act as a professional web developer with experience in the frameworks and technologies we were using, thus making sure that we would obtain technically sound solutions. Using guided reasoning, we were able to reach a complete high-level understanding of the application code, making sure that every design decision and implementation choice were properly explained and documented.

4.1.7 Professional identity and way of working

Working with AI helped us improve our critical evaluation skills, as we found ourselves in need of analyzing and evaluating code that was not written by us, but that we were going to claim as ours. Additionally, as we strived to make our initial prompt as detailed as possible to minimize the room for error, this approach allowed us to develop a more complete and detailed design of the final product before we started coding. However, we do not have complete mastery of every small implementation detail as we would if we had written the code ourselves. This is due both to the limited time we had to deliver the code and to our limited expertise with the frameworks we used.

4.2 Attached AI Interaction Log

4.2.1 iRE agent

Below we include the log of the interaction with the iRE agent on InnSpill. This interview transcript was also included in task 1.1.

Can you provide a general description of the application you would like to create?

The application we aim to create is the AI Guidebook for Students, designed to support academic work by helping students use generative AI tools responsibly. It will provide features such as AI usage logging, assignment-level AI declarations, ethical guidelines, and automated compliance checks with institutional policies. The system will also include a personal dashboard for visualizing usage patterns, ensuring transparency and privacy while promoting academic integrity.

Can you describe specific, measurable ways you'd want to track your AI usage that would be helping students use generative AI tools responsibly?

To track AI usage effectively, the system shall implement several measurable methods. First, it should log the frequency of AI tool usage per assignment, allowing students to see how often they rely on AI assistance. Second, it shall require students to declare the specific AI tools used for each assignment, promoting transparency. Third, the system can provide visual analytics on usage patterns over time, highlighting trends in AI dependency. Lastly, automated compliance checks against institutional guidelines should be included to ensure responsible usage aligned with academic integrity standards. Each of these elements can be quantified and analyzed to foster responsible AI tool use.

How would the assignment AI declaration be performed?

The assignment AI declaration shall be performed through an integrated form within the application. Students will be prompted to complete this form upon submission of each assignment, specifying the AI tools utilized. The system shall require the following fields: assignment title, AI tools used, purpose of AI assistance, and any generated content. This structured approach facilitates transparency and ensures compliance with academic

integrity policies. Additionally, the declaration process should be straightforward to encourage consistent usage by students.

Should the AI guidelines compliance checks be done periodically? If relevant, what would be the frequency of these checks?

The system shall implement periodic compliance checks to ensure adherence to AI guidelines. The frequency of these checks could be set to occur at key milestones, such as after assignment submissions or at the end of each academic term, depending on institutional policies. It's essential to balance the frequency with the need for timely feedback and support for students in maintaining academic integrity.

What do you mean by "balancing the frequency" of the checks?

Balancing the frequency refers to finding an optimal schedule for compliance checks that provides timely feedback to students without overwhelming them. Too frequent checks may lead to unnecessary stress or administrative burden, while infrequent checks could result in missed opportunities for students to correct their AI usage and maintain academic integrity. It's crucial to establish a frequency that supports effective learning while ensuring compliance with guidelines.

Should these check also evaluate sustainability in the use of AI?

The system shall consider including sustainability evaluations as part of the compliance checks for AI usage. Evaluating sustainability would ensure that students are not only using AI responsibly but also being mindful of the environmental impact associated with these technologies. This could involve assessing the frequency and duration of AI tool usage and encouraging efficient practices in alignment with broader institutional sustainability goals.

Describe a typical workflow of a student using the application. How would the user interact?

A typical workflow for a student using the application may include the following steps:

1. Login: The student logs into the application and accesses their personal dashboard.
2. AI Tool Selection: When starting an assignment, the student selects from a list of AI tools they plan to use.
3. Assignment Declaration: The student completes an AI declaration by providing details about the tools used and a brief description of their contributions to the assignment.
4. Reflection: The system prompts the student to reflect on their AI usage regarding academic integrity, providing questions to consider.
5. Submission: After completing the assignment, the student submits it along with the AI declaration, which is logged for compliance checks.

6. Feedback and Compliance Check: The system periodically evaluates the AI usage in relation to guidelines and provides feedback, allowing the student to adjust their practices as necessary.

7. Review and Adjust: The student can review their usage patterns over time on their dashboard, making adjustments based on feedback and sustainability prompts.

This workflow ensures that students engage with the application in a structured manner while promoting responsible AI usage and compliance with academic integrity policies.

What details should the student provide during the assignment declaration steps?

During the assignment declaration steps, the student shall provide the following details:

1. AI Tools Used: A list of specific AI tools utilized during the assignment.

2. Description of Usage: A brief explanation of how each tool contributed to their work, including tasks supported by the AI.

3. Reflection on Academic Integrity: Insights on how their AI usage aligns with institutional academic integrity policies, addressing any potential ethical concerns.

4. Duration of Use: An estimate of the time spent using each AI tool during the assignment process.

5. Sustainability Considerations: Any efforts made to use AI tools efficiently and sustainably, if applicable.

These details will help ensure transparent and responsible use of AI in their academic work.

Can you make an example of "efficient and sustainable" AI usage?

An example of "efficient and sustainable" AI usage could be a student using an AI writing assistant to draft an essay. Instead of relying solely on the AI for the entire content, the student uses it to generate outlines and key points, then integrates these suggestions with their own research and writing. This approach minimizes excessive reliance on AI, reduces computational resources, and promotes critical thinking. Additionally, the student could limit AI usage to specific phases of the writing process, such as brainstorming and editing, thereby optimizing the time spent with the tool while being mindful of its environmental impact.

What tools should the list of AI models available for usage declaration include?

The list of AI models available for usage declaration should include:

1. ChatGPT: For conversational assistance, brainstorming, and drafting text.

2. GitHub Copilot: For code generation and programming support.

3. Claude: For natural language processing tasks, including summarization and content generation.

4. Grammarly: For grammar checking and writing enhancement.

5. Turnitin: For plagiarism detection and ensuring originality in writing.

6. DALL-E: For generating images and visual content related to assignments.

7. Data analysis tools (e.g., Tableau, Python libraries): For data visualization and analysis in research projects.

Inclusion of these tools will help ensure comprehensive tracking of AI usage across various academic tasks.

Should the student be given the opportunity to declare models that are not present in this list?

Yes, the system shall provide students the opportunity to declare AI models that are not present in the predefined list. This flexibility is essential to accommodate the diverse array of AI tools available and ensure that all forms of AI usage are transparently documented. Students should be able to specify the tool name, its purpose, and how it contributed to their assignment, promoting a comprehensive understanding of AI engagement in academic work.

What points should the "guided reflection" address? Should it be dependent on what the student previously declared?

The "guided reflection" should address the following points:

1. Purpose of AI Usage: Why the student chose to use specific AI tools for the assignment.

2. Impact on Learning: How the AI tools influenced their understanding and learning process.

3. Ethical Considerations: Any ethical concerns related to AI usage, including potential biases or misrepresentation of work.

4. Alignment with Academic Integrity: How their use of AI adheres to institutional policies and academic integrity standards.

5. Sustainability Practices: Any efforts made to use AI tools efficiently and sustainably.

Yes, the reflection prompts should be tailored based on the student's previous declarations to ensure relevance and encourage deeper engagement with their specific AI usage. This personalization will enhance the quality of their reflections.

Can you describe more clearly how the student should perform the "review and adjust" step basing on the feedback received?

During the "review and adjust" step, the student shall follow these actions based on the feedback received:

1. Access Feedback: The student logs into their dashboard to review the compliance feedback provided by the system regarding their AI usage.

2. Analyze Feedback: The student examines specific areas highlighted in the feedback, such as ethical concerns, sustainability practices, or adherence to academic integrity guidelines.

3. Identify Improvement Areas: Based on the feedback, the student identifies areas where their AI usage can be enhanced, such as reducing reliance on AI for certain tasks or improving their reflection on ethical implications.

4. Adjust Practices: The student modifies their approach to using AI tools in future assignments, implementing strategies to address the identified improvement areas. This may include setting personal goals for responsible AI usage or exploring additional resources on academic integrity.

5. Document Changes: The student notes these adjustments in their usage logs, ensuring a record of their commitment to continuous improvement in AI engagement.

By following these steps, the student can effectively respond to feedback and strengthen their academic practices.

What is the scale of this application? how many students are we expected to serve?

The application is designed to support a large-scale student population, ideally serving thousands of students across various academic programs. Given the increasing integration of AI tools in education, the platform should be scalable to accommodate user growth, enabling seamless access and functionality for all users while maintaining performance and compliance with institutional guidelines. Specific student capacity metrics may depend on institutional partnerships and anticipated adoption rates.

What are the guidelines for data storage and data privacy?

The guidelines for data storage and data privacy in the application shall include:

1. Data Minimization: Only collect and store data that is necessary for functionality and compliance purposes, avoiding excessive data collection.

2. Anonymization: Where possible, anonymize personal data to protect student identities and ensure privacy during data analysis and reporting.

3. Secure Storage: Implement strong encryption methods for data at rest and in transit to safeguard against unauthorized access and breaches.

4. Access Control: Limit access to personal data to authorized personnel only, ensuring that data handling complies with institutional and legal standards.

5. Compliance with Regulations: Adhere to relevant data protection regulations (e.g., GDPR, FERPA) to ensure students' rights regarding their data are respected and upheld.

6. Transparency: Provide clear information to students about what data is collected, how it is used, and their rights concerning their data.

These guidelines will help ensure that the application maintains high standards of data privacy and security while supporting responsible AI usage.

Could you clarify better the requirements about data privacy?

The data privacy requirements for the application shall include:

1. User Consent: The system shall require explicit consent from students before collecting personal data, ensuring they understand the purpose and scope of data usage.

2. Data Access Rights: The system shall provide students the ability to access, update, or delete their personal data at any time, in compliance with applicable regulations.

3. Data Retention Policy: The system shall implement a data retention policy that defines how long personal data will be stored, ensuring data is only retained as long as necessary for compliance or operational needs.

4. Incident Response Plan: The system shall have a documented incident response plan to address potential data breaches, including notification procedures for affected students.

5. Regular Audits: The system shall conduct regular audits of data handling practices to ensure compliance with privacy regulations and internal policies.

6. Privacy Training: All personnel handling personal data shall receive training on data privacy practices and institutional policies to ensure responsible data management.

These requirements ensure that the application prioritizes student privacy and complies with legal standards, fostering trust and accountability.

What are the requirements on usability?

The usability requirements for the application shall include:

1. User-Friendly Interface: The system shall provide an intuitive and accessible user interface that allows students to navigate easily and perform tasks without unnecessary complexity.

2. Responsive Design: The application shall be optimized for various devices (desktop, tablet, mobile) to ensure a consistent user experience across platforms.

3. Clear Instructions and Help Resources: The system shall offer clear instructions and help resources, including tooltips and FAQs, to assist students in understanding how to use the application effectively.

4. Feedback Mechanisms: The application shall incorporate mechanisms for students to provide feedback on usability issues, enabling continuous improvement based on user experience.

5. Minimal Learning Curve: The system shall be designed to minimize the learning curve for new users, allowing students to quickly become proficient in using the application.

6. Accessibility Compliance: The application shall adhere to accessibility standards (e.g., WCAG) to ensure that all users, including those with disabilities, can effectively use the platform.

These requirements aim to create a seamless and supportive environment for students, enhancing their engagement with AI tools and promoting responsible usage.

Are there any other users that would be using the application, apart from the students?

The primary users of the AI Guidebook are students, but other potential users may include educators and academic administrators. Educators might use the application to monitor AI usage patterns among students and provide guidance, while academic administrators could leverage the data for compliance and policy enforcement. Identifying their specific requirements will be crucial for capturing a comprehensive set of functionalities.

What are the requirements about data security?

The system shall ensure that all user data is encrypted both at rest and in transit to protect against unauthorized access. The system shall implement role-based access control to restrict access to sensitive data based on user roles. Additionally, the system shall log all access and modification activities to support auditing and compliance checks.

Other than a login page, which forms of access control should the system implement?

The system shall implement multi-factor authentication (MFA) to enhance security during user login. Additionally, the system shall provide session management controls, ensuring users are automatically logged out after a period of inactivity. Fine-grained access control should also be enforced for different features, allowing users to only access functions related to their roles.

What are the requirements about storage performance and API response time?

For storage performance, the system shall ensure data retrieval times do not exceed 200 milliseconds for logged AI usage data. Regarding API response time, the system shall

provide responses to user requests within 300 milliseconds under normal operating conditions. These performance metrics are critical to maintaining a smooth user experience.

What do you mean by "normal operating conditions"?

"Normal operating conditions" refer to the typical usage scenarios of the application, where user load and system resources are within expected parameters. This encompasses standard traffic patterns, average data volume, and typical user interactions. It's important to define these conditions clearly to ensure performance metrics are achievable and realistic.

Can you define these conditions more precisely?

Normal operating conditions shall be defined as follows:

1. User Load: Up to 100 concurrent users accessing the application simultaneously.
2. Data Volume: The system shall handle an average of 10,000 logged AI usage records at any given time.
3. Interaction Type: Standard user activities include logging AI usage, querying usage patterns, and accessing ethical guidelines.

These parameters ensure that performance metrics remain realistic and achievable during typical usage scenarios.

How is the system expected to behave in abnormal conditions (for example, excessive request load)?

The system shall implement performance monitoring to identify and manage excessive request loads. It shall include load balancing mechanisms to distribute requests evenly across servers and maintain responsiveness. In case of peak load situations, the system shall provide users with feedback about potential delays and may implement rate limiting to ensure fair access. Additionally, the system shall log abnormal conditions for further analysis and improvement.

4.2.2 GitHub Copilot

All code generation and review tasks were carried out on GitHub Copilot, using Claude Sonnet 4.5.

Code generation

Below we include the log of prompts and responses from our interaction with GitHub Copilot for the purpose of code generation.

The following is the prompt we used to generate the full code of the application. It was also reported in task 2.1.

> "Role description:
> You are a senior full-stack software engineer with experience with

- Frontend: HTML, CSS, TypeScript, React
- Backend: Node.js, Express
- Databases: PostgreSQL, MySql
- Software engineering practices: clean code, modular design, security best practices

Here is a description of the project context.

We are building a "AI Guidebook System" web application to support students in everyday academic work, when generative AI tools are embedded in everyday study practices. We aim to provide clear guidance that helps them use AI responsibly. The application will provide the opportunity to declare AI usage in assignments to promote transparency. This will be done through a form in which the student will declare which AI tools he has used and for what purpose. The data collected through the form will be stored in a database.

Act like a professional developer working in a real team. Respond and help me design and implement my web application. When you help me, there are some guidelines that you should follow.

- You should always explain your reasoning briefly before writing code, thinking step by step.
- When you are producing code, you should always produce only one file of code at a time and comment them properly.
- Produced files should have a clearly stated file name at the top (e.g., '/server/index.js').
- If you miss any information, ask for clarification before coding.
- You should not skip any steps in the development process.

Technical requirements:
The web application should use React and TypeScript for the frontend, as well as Node.js and Express for the backend and MySQL for database.

Application description:
We are focusing on the assignment declaration form of the application. The frontend should consist of two pages, one containing the form to be filled out by the user and the other contain a home page that will load at the start of the system and after the form is submitted. The home page will contain all forms submitted, as well as the "create new form" link. When starting the application, the landing page is the homepage with already submitted forms from the database. After the submission of the form, the user is redirected to the homepage where the newly submitted form is present.

User stories:
The requirements for the assignment declaration form feature are listed below.

- As a student, I want to declare AI usage in assignments by filling out a form, so that I can ensure compliance with the ethical guidelines provided by the institution.
- As a student, I want to specify specific tools used in each assignment and their purpose, so that I can achieve transparency in my work.
- As a student, I want to be able to select the tools I used during assignment declaration from a list (ChatGPT, GitHub Copilot, Claude, Grammarly, Turnitin, DALL-E, data analysis tools), so that I can perform the declaration quickly.
- As a student, I want to have the option to declare AI tools that are not present in the list during assignment declaration, so that I can feel free to use the tools that are most useful to me while still being transparent.
- As a student, I want to declare the content generated by AI in my assignments, so that I can be transparent in my work.
- As a student, I want to attach screenshots or examples of AI-generated content to my declaration, so that I can provide clear documentation.
- As a system admin, I want the system to handle an average of 10,000 logged AI usage records at any given time. so that I can perform meaningful statistics.
- As a system admin, I want the system to store the data in a database, so that I can easily retrieve, update and delete information.

Additionally, the code should satisfy the following technical requirements:

- The image files uploaded through the form should be stored in a directory on the server filesystem. The database should store a path to the file together with the rest of the usage record.
- The system should not allow the user to upload files that are not images or that are larger than 5MB in size.
- The text field for the "Other" option in the AI tool selection in the form should allow only strings of letters (lowercase and uppercase), numbers and punctuaction of maximum length of 50 characters.
- The database should include one table with the following schema:

```
CREATE TABLE IF NOT EXISTS ai_declarations (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_name VARCHAR(255) NOT NULL,
  assignment_title VARCHAR(255) NOT NULL,
  ai_tool VARCHAR(255) NOT NULL,
  usage_purpose TEXT NOT NULL,
  ai_content TEXT NOT NULL,
  screenshot_path VARCHAR(500),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_user_name (user_name),
```

```
  INDEX idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_c
```

Define a structured development strategy for the application and proceed with the code development, ensuring all requirements and guidelines are followed. The result should be a working full-stack web application."

The response to this prompt was the collection of the code files for our application. We do not report it in full as it is very long. The response was structured as follows:

- Development strategy
  - Phase 1: Project Setup Architecture
    - Define the project structure
    - Set up the database schema
    - Initialize the project with necessary dependencies
  - Phase 2: Backend Development
    - Set up Express server
    - Create database connection and models
    - Implement API endpoints (GET all declarations, POST new declaration)
    - Add file upload handling for screenshots
  - Phase 3: Frontend Development
    - Set up React with TypeScript
    - Create the Home page (list of declarations)
    - Create the Declaration Form page
    - Implement routing between pages
    - Add form validation and submission logic
  - Phase 4: Integration Testing
    - Connect frontend to backend APIs
    - Test the complete flow
    - Add error handling
- Technical architecture
  - Database structure: description of the schema in SQL, coherent with what we specified in the prompt
  - Project structure: overview of the file structure for the project
  - API endpoints:
    - `GET /api/declarations` - Fetch all declarations
    - `POST /api/declarations` - Create new declaration(s)
- Project files: there followed all the project files, consisting of file name and location, code and explanation of reasoning, for a total of 20 files.

Then, after downloading the code and importing it into our local workspace, we asked for a setup tutorial, because some files had to be created manually. Copilot specified the project structure, each file that needed to be created, how to install the dependencies, setup of the MySQL database, and finally how to run the application.

Then we requested a general explanation of the application. The prompt we used is the following:

> "Explain the overall functioning of the application. What are the components? What is their logical and functional grouping? How do they interact with each other? Describe the process of the assignment declaration: what happens when a user submits the form? What are the components involved?"

The answer, which is summarized in task 2.2, included an overview of the application architecture, including the three layers (Presentation/Application/Data), the structure of the client and server components and the purpose of each file. Additionally, the response provided a logical grouping of the components by layer and by functionality. Finally, a complete step-by-step description of the form submission data flow was provided, including the called functions, the network requests and responses and the generated objects. The response also included an interaction diagram and an explanation of key design decisions.

Code review

Here we include again the prompts and responses we used to identify code smells in task 2.3.

Dead or unreachable code

> "Help me identify dead or unreachable code in the codebase. Follow this reasoning process: examine both the client and server code, and find any functions or components that are unreachable from the rest of the code and so can be removed. Report them one by one and explain why their are unreachable and what their intended purpose was. Recommend an approach to fix the problem. If there are no dead or unreachable code snippets, report that the code does not exhibit this smell."

This prompt elicited the following response from the model:

> "Dead Code Analysis Report - Repository: Raffaella-Corcione TDT4242_group03 - Executive Summary
> Good News! Your codebase is very clean with minimal dead code. The commit message "fixed server response inefficiency" indicates you've already addressed the main issue I would have flagged. I found 2 instances of unused/dead code:
>
> - Unused Type Export - Minor
> - Unused CSS Utility Class - Minor
>
> 1. Finding 1: Unused Type Export AITool - Location: `files/client/src/types/index.ts (L`
>   Why It's Dead Code: This type is exported but never imported or used anywhere in the codebase. Evidence:

- AI_TOOLS constant is used in DeclarationFormPage.tsx (line 5, line 256)
- AITool type is never imported in any file
- No references to AITool type in the entire codebase

Intended Purpose: The AITool type was meant to provide type-safe references to AI tool names, restricting variables to only valid predefined tool names from the AI_TOOLS array. Impact:

- Runtime: None (types are erased at compile time)
- Bundle Size: None
- Maintainability: Minor - having unused exports can be confusing

2. Finding 2: Unused CSS Utility Class .text-center - Location: `files/client/src/index.css` (Lines 103-105). Why It's Dead Code: This CSS class is defined but never used in any component. Evidence - Searched all component files: components use inline styles or specific classes. Intended Purpose: Utility class for quickly centering text (Tailwind CSS style). Common pattern in modern CSS frameworks. Impact:
   - Runtime: Minimal (just a few bytes in CSS bundle)
   - Bundle Size: 40 bytes
   - Maintainability: Low - unused utility classes clutter the codebase
3. Finding 3: Potentially Unused Response Field count - Location: `files/server/routes/decl` (Line 61). Why It Might Be Dead Code: The count field is sent to the client but never directly used. Intended Purpose: Provide metadata about the result set (common REST API pattern). Impact:
   - Runtime: Minimal (one number calculation)
   - Network: 10-15 bytes per request

   Is This Actually "Dead"? Not quite! This is a gray area:
   - It's a best practice for REST APIs to include metadata
   - It's useful for debugging/monitoring
   - If you add pagination later, you'll need it.

   Final Verdict: Your codebase does NOT significantly exhibit the "dead code" smell. Overall Code Health: 9.5/10 Strengths:
   - Clean, well-structured code
   - All major functions and components are actively used
   - You've already fixed the main inefficiency (redundant SELECT query)
   - Good separation of concerns
   - Proper error handling throughout
   - No unused imports or large unused functions

   Minor Improvements: Consider using the AITool type for better type safety and remove the unused .text-center utility class."

Note: the response included also the involved code and suggested fixes. We do not report those for brevity, as the approach to fix the code smells has already been discussed in task 2.3. The same applies for the following interactions.

Duplicated logic

"Help me identify duplicated logic in the codebase. Follow this reasoning process: examine both the client and server code, and find any functions or components that implement the same logic and can be simplified. Report them one by one and explain what code is duplicated and the reasoning behind it. Recommend an approach to unify and centralize the duplicated logic. If there are no duplicated code snippets, report that the code does not exhibit this smell."

This prompt elicited the following response from the model:

"Duplicated logic Analysis Report - Repository: Raffaella-Corcione TDT4242_group03 - Executive Summary
I've identified 5 instances of code duplication in your codebase. These range from minor (repeated validation patterns) to significant (error response structures). While the codebase is generally well-structured, consolidating these duplications would improve maintainability and reduce the risk of inconsistencies.

1. Finding 1: Duplicated Validation Logic for Required Fields HIGH PRIORITY - Duplicated Between: Client and Server - Location 1: Client-side Validation (DeclarationFormPage.tsx lines 106-128), Location 2: Server-side Validation (declarations.js lines 84-94). Why This is Duplicated: Both client and server validate the same business rules. The Problem:
   - Maintenance burden - If requirements change (e.g., add minimum length), you must update two places
   - Inconsistency risk - Client might allow something server rejects, or vice versa
   - Error message mismatch - Client shows specific errors, server shows generic message

   Reasoning: This duplication exists because:
   - Client validation = User experience (instant feedback)
   - Server validation = Security (never trust the client)

   Both are necessary! But they should share the same rules. Recommended Solution: Create a shared validation schema that both client and server can use. Benefits:
   - Single source of truth for validation rules
   - Consistent error messages on client and server
   - Automatic type safety
   - Easy to add new rules (add once, works everywhere)
2. Finding 2: Duplicated `.trim()` Calls MEDIUM PRIORITY - Location: DeclarationFormPage.tsx. The Problem:
   - Performance - Minimal impact, but wasteful

- Code noise - Clutters the logic
- Maintenance - If you add a field, you must remember to trim everywhere

Reasoning: This duplication exists because form state stores the raw (untrimmed) values, so you must trim on every use.

3. Finding 3: Duplicated Error Response Structure MEDIUM PRIORITY - Duplicated Across: All server error handlers. Why This is Duplicated: Every error response follows the same structure but is manually constructed each time. The Problem:
   - Inconsistency risk - Easy to forget a field or use wrong structure
   - Maintenance - If you want to add error codes or stack traces, update multiple places
   - Testing - Hard to ensure all errors follow the same format
4. Finding 4: Duplicated File Validation Logic LOW PRIORITY - Duplicated Between: Client and Server. Why This is Duplicated: Both validate
   - File must be an image
   - File must be ¡ 5MB

   The Problem:
   - Magic number duplication - 5 * 1024 * 1024 appears in both places
   - Different validation approaches - Client checks MIME type prefix, server checks extension + MIME
   - Inconsistency risk - Easy to change limit on one side but not the other
5. Finding 5: Duplicated Button Styles LOW PRIORITY - Location: Multiple CSS classes with similar patterns. Why This is Duplicated: Button styles are defined in both HomePage.css and DeclarationFormPage.css with slight variations. The Problem:
   - Inconsistency - HomePage uses display: inline-block, DeclarationFormPage uses display: inline-flex
   - Maintenance - Changing button styles requires updating multiple files
   - Bundle size - Duplicate CSS rules increase file size

   Overall Assessment: Code Duplication Score: 6/10 (Lower is better). Findings:
   - Good: No massive copy-paste blocks
   - Good: Components are well-separated
   - Moderate: Validation logic is duplicated (expected but could be improved)
   - Minor: CSS and utility patterns repeated

   Priority Actions:

- High: Implement shared validation schema (Finding 1)
- Medium: Centralize error responses (Finding 3)
- Medium: Create .trim() helper (Finding 2)
- Low: Extract button styles to global CSS (Finding 5)
- Low: Share file validation config (Finding 4)

Conclusion: Your codebase exhibits moderate code duplication, which is common in full-stack applications where client and server need similar logic for different purposes (UX vs security). The most impactful improvement would be implementing shared validation schemas (Finding 1), as this addresses the highest-risk duplication and improves both developer experience and user experience."

Since we decided to implement the fix for the error handling duplication, we also asked to provide the needed modifications. The model responded with the new code and the precise location where the change needed to be applied.

Testing

We employed AI for generating our automated unit tests and help set up the testing environment. As an example we report the prompt we used for generating test case n. 1. All the others are very similar and follow the same pattern.

> I want to write an automatic unit test case to test the validateForm() function in the file files/client/src/pages/DeclarationFormPage.tsx. The test should assert the normal functioning of the form input validation, so the function should return the value true. The test should have the following form input data:
>
> name: Ola Nordmann Assignment title: assignment 1 AITools: ChatGPT Other tool: (none) Purpose: summary of lecture contents Content: this is an example content. Screenshot: (none)
>
> Generate a file that contains the code for this testcase, so that it is completely executable automatically.

The model generated a file (`files/tests/DeclarationFormPage.validateForm.happy-path.test`) with the complete code of the test. The test did run correctly and there was no need for modifications.